

LP2B – Multimedia: Digital Representation

- LP2B Practicum Class n°3 -

Final Project : the 3-in-1 App !

Putting it all together with transfers and a menu

This Practicum is about applying knowledge from ALL the Tutorials

What are we going to Make ?

- We want to regroup **THREE** games within a single Unity App
Two previous ones (Bricker Breaker, Apple Catcher) , and a new one (Mini UFO Attack)
- The app initializes with a menu that has **3 buttons**. Clicking one of those buttons will initialize the matching game by loading the corresponding scene
- With this menu being part of the app, we will need a form of navigation, meaning the following features will have to be added to all of our games :

- Pushing the « Escape » key during any of them will return to the main menu
- When a game ends, a game over screen with a menu appears
- Game Over menu allows to either replay the same game, or return to main menu

- The new Game, a retro 2D Shoot-them-up, has to be coded by you entirely, and comes with its own list of requirements
- Don't worry : A lot of content will be recycled from previous classes, so this isn't as much work as it could initially seem *(plus, you MUST be working in groups of at least 2)*



A Menu...



...To access 3 games !

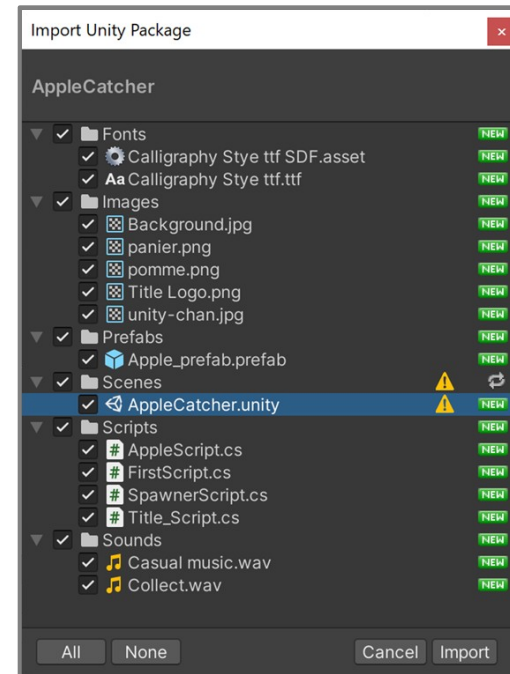
The Video Demo that sums it Up

- Because a very detailed description of the features would be very long and tedious, this powerpoint works in tandem with a little demo video of a completed project I made myself
- You can see it here : <https://youtu.be/Z5gFSpMxhjl>
- The video contains some text explanations that I edited over the footage, but its main purpose is to « lead by example » by showing you the app running
- You are NOT meant to make a perfect copy of the app in the video. That would require a bunch of things we didn't see together like particles and masks. *(but if you can do it through research, you'll of course get an amazing grade!)*
- The goal of this powerpoint is, in good part, to explicit what part of this video you MUST reproduce in your work
- But first, let's talk about how you can **transfer your previous work** on Apple Catcher and Brick Breaker to this new project...



Importing a Pre-Made Unity Package

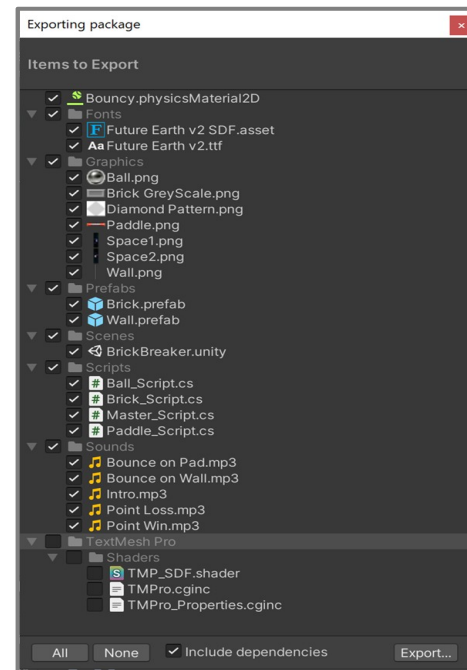
- Unity allows the transfer of data between projects with **Asset Packages**
Those packages are not just about code. They can also contain assets, scenes...
- Just in case : I already prepared a package with the TD « Apple Catcher » in
If you couldn't do everything during TD, this will allow you to catch up
- To import the file : **(Assets => Import Package => Custom Package)**
- Unity will then show you the contents of the package, allowing you to review what you're about to import.
You can even choose to only import parts of the package. **Here, please import all (default)**
- A warning should display in the Console. Don't worry, it's just because the Scene you imported was « number 0 » in the original project
And the project you have here probably also has a scene 0. The conflict is solved automatically, but you'll have to add the « Apple catcher » scene to the build settings menu, in order to give it a new number



*The import Window allows you to review what you are about to import.
Unity Notifies you of any conflict or potential problem with warnings and/or errors*

Exporting Brick Breaker as a Package

- You can **create your own packages** to export any part of a project you want to
It's possible to export everything, or just certain parts selectively
- Make an export of your own version of « Brick breaker » done in TP
Export the **WHOLE ASSET FOLDER** of the old project, except **textMeshPro** in your package
- Import your package into « big final project », like we did for Apple Catcher
You should likely get the same warning for the scene. But this is fine
- If you have problems, it's probably because some entities in your different projects share the same name
The package manager will point that problem out and help you to solve it
- Since you'll be **working in groups**, pick the best Brick Breaker among those created individually, or better : try to merge their best qualities into one !
- If you have trouble with packages, the **brute force solution** is to copy-paste the files in the new project. *(But this is more likely to cause issues. Try to use packages.)*



*The Package export Window allows you to review what you are about to export
TextMeshPro doesn't have to be part of your export : it's added automatically by Unity when needed*

The Game Selection Menu

- A key part of this final project is the **GAME SELECTOR MENU**

There, you will use **Video, Canvas and Animations**. The big concepts of the last tutorials

- Requirements for this menu :

- 1) The « planets » are, in fact, buttons. Clicking one loads the corresponding game
- 2) The Planet buttons **MUST** use the Unity Button class. Meaning they must also be part of a Canvas
- 3) The Planet Buttons use **animations** so the user can easily understand that they are interactive
- 4) The background is a video. Make sure it will **ALWAYS** appear behind the planets
- 5) The video and images necessary to make this menu are all available on Moodle
- 6) You are free to add extra features here, if you want.
(might not be the most inspiring place to do so, though)

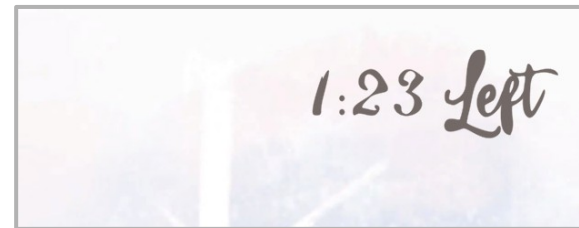


Extra features for « Old Games »

- I want you to **add a few extra things** to Apple Catcher and Brick Breaker
As, right now, they aren't exactly real games, having no end condition

FOR APPLE CATCHER :

- Add a time limit. When time is over, the game shows the game over menu
- Time should be displayed in a natural format for time, like 1:00



Example for the Timer you must add in Apple Catcher

FOR BRICK BREAKER :

- Add a lives system. When the ball falls below, the player loses a life
- When all lives are lost, the game shows the game over menu
- You can display the remaining lives in several ways (text, icons, etc...)



Example of an icon-based life counter in Brick Breaker

The features above are **MANDATORY**.

But you can also add as many extra features as you want, if you believe it improves the experience ! Be creative !

The New Game : Mini UFO Attack

- While Apple Catcher and Brick Breakers are direct extensions of what we did in TD and TP respectively, you are expected to make one **entirely new** game
- Mini UFO Attack is a **retro 2D shoot-them-up**, like R-Type or Galaga
...Space invaders is a bit TOO retro to be a relevant comparison to what we want
- **Mandatory Actions/abilities for the player** : they can move in all 4 directions and shoot a « machine gun » type weapon as well as a bonus « super » weapon
But you have the freedom to add extra game mechanics/controls if you want to. In fact, it's needed if you want to get full marks !
- **Different enemies** appear to try to shoot or ram the player. There has to be at least 4 types of enemies in your game
But I provide you with enough graphics to make 6 of them if you want !

BTW : There is a Unity Package on Moodle that contains the Graphics for this game, as well as a few prefabs you may use to save time



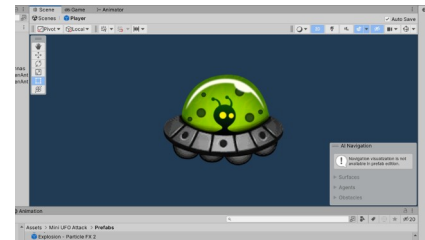
R-type, one example of the many, many Shoot-them-up games of the 90's



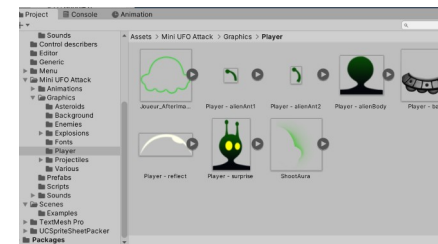
The clone you're about to make

Mini UFO : the Player

- The Package I am providing you contains a **prefab for the player**, which is made of many moving parts for the sake of animation
- **You have to add custom script(s)** to it to give it at least those behaviors :
 - 1) Can move using arrow keys, but not outside the visible playing field
 - 2) Can shoot a « machine gun laser » by holding down a key (probably spacebar)
 - 3) Gets damaged when hitting an enemy bullet or an enemy. This causes a loss of shield (health). If health didn't reach zero, player becomes invincible for 1 to 2 seconds.
 - 4) Represent this temporary invincibility visually in some way (sprite blinking like in my demo, color change, sprite change...)
 - 5) Player has a « special attack » (bomb) that is limited in use. It is triggered with a specific key (*unless you have a wild idea, « B » key is the easy choice*)
 - 6) You have some freedom in how the bomb works exactly. But just make sure its power matches its limited availability (*removing all enemies and bullets is the classic effect*)
 - 7) Player can collect the powerups that exist in the game to gain benefits



*The Package contains a prefab for the player
It's made of a bunch of separate sprites
Nested together (for animation purposes)*



*The package contains graphics you can use for
the death animation and other effects*

Importing the package also imports its folder structure. If you don't like it, feel free to reorganize !

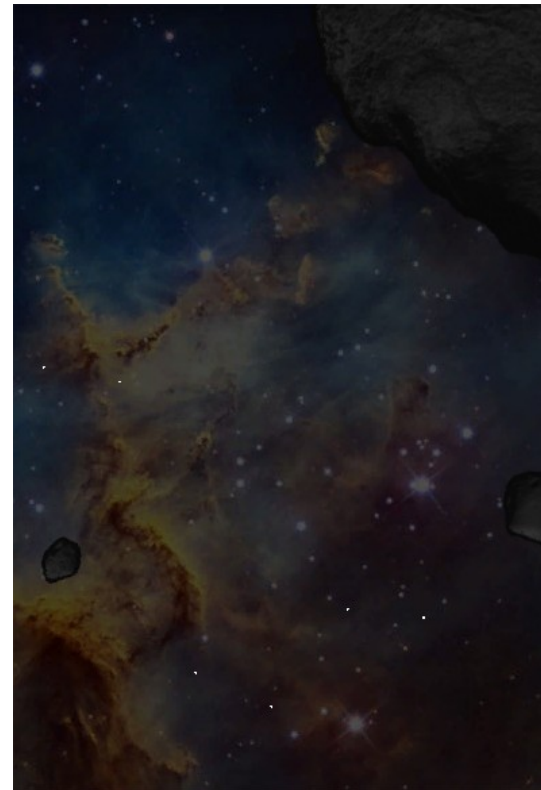
Mini UFO : The Enemies

- A shooter game needs armadas of disposable enemy ships to shoot down !
The package contains sprites for 6 of them, and for several « typical » attacks that can be seen in the demo video (shots, big laser, missile)
- You are **required to implement 4 enemies** and at least **2 types of projectiles**, and by that I mean : they have to behave differently and not just be « reskins »
- The video demo can be an obvious source of inspiration on what to do. But you can do pretty much **anything that makes sense in a game like this**
 - Enemies can try to ram the player, or stick to the right of the screen to shoot.
 - Enemies that shoot very different projectiles or shoot a « common » projectile in a very different and specific way count as « different enemies »
 - Projectiles that move in straight line, in sinusoidal pattern, that pursue the player...
 - « Bosses », aka super strong enemies that fight alone (or with only a few « randos » around) do not contradict those rules and are therefore acceptable
- Don't forget : killing enemies has a probability of spawning a Powerup
(you should probably have different drop rates depending on enemy strength)



Mini UFO : Interface, Background

- Interface is pretty self explanatory by looking at it, but to clarify :
 - Interface shows the remaining shield (health) using separate sprites. A small algorithm will be needed to convert « health as an int » into such a display
 - Interface shows the number of available « bombs ». You can use icons like I did or use a text-based display if you prefer. There is a max number of bombs (of your choice)
 - Interface displays score, which increases by destroying enemy ships. Enemies have different point values depending on their difficulty/strength.
- The background gives the illusion of movement forwards by :
 - Randomly spawning asteroid prefabs on the right, outside view of the camera
 - Those prefabs are programmed to move left at randomized speed and rotation speed, and self-destruct when they exit the screen on the left
 - Asteroids shouldn't use Rigidbody, and move through their code modifying their « transform » properties (transform.position, etc...)



Mini UFO : Powerups, Music, Sound

- There are **two powerups in the demo**, shield and bomb :
 - Shield allows player to heal, bomb gives them one « mega bomb » ammo to use
 - I also provide you with graphics for other powerups, unused in my demo. You can use them to implement different/extra powerups if you want to (*attack upgrade, etc.*)
 - Powerups that are different will count towards « innovation points », especially if they are more complex (*like giving the player a different base weapon, for instance*)
- **Music and Sound** are better explained by the video than by a written text. But here are some things you can try to emulate :
 - Sometimes, a « musical pause » or « silence pause » can allow the player to breathe, and enhance the feel of the game
 - Try to manage the volumes so that all parts of the app are « about as noisy »
 - Demo shows an «almost ideal result », you don't have to go this far for a good grade



Make the App better, your way !

While there are many features explicitly requested, this class takes the approach of giving you a **lot of freedom**. You have the option to add your own ideas to the minigames, as long as they don't contradict the things explicitly requested.

I am also aware that **Mini UFO attack is more demanding than the « 3rd game » of the previous years**. ...But your « additions » can also happen in the other, comparatively simpler games !

EXAMPLES :

- Adding powerups and « poison apples » to Apple Catcher
- « Multi-ball » or « Sticky paddle » powerups in Brick Breaker
- Multiple weapon/bomb types in Mini UFO attack
- « Bullet time » ability in one or more game
- Limiting abilities through a regenerating meter instead of powerups
- Using particle effects or other methods to improve graphics
- Etc, etc...



Initiative is part of the work!

About 1/5th of the grade will be dedicated to your additions and innovations, going BEYOND the mandatory features described previously.

Returning to Game Select Menu

- To really tie our App together, we want give the user the ability to **exit a minigame at any time**, by pressing « Escape »
This will allow our player to try a game, and then another, without having to exit the program
- Doing this is **not any different** than what you had to do for the menu : you will need to use a variant of the « Scene loading Coroutine »
This particular Coroutine was given to you explicitly in tutorial class. You can just copy it !
- Just remember to implement this functionality in **all 3 minigames**
I don't mind if there are several copies of the coroutine in different scripts
- Pressing « Escape » on the menu must exit the application (build only)
Closing the application doesn't work in test mode, as it doesn't run independantly then
- The line of code to close the application is :

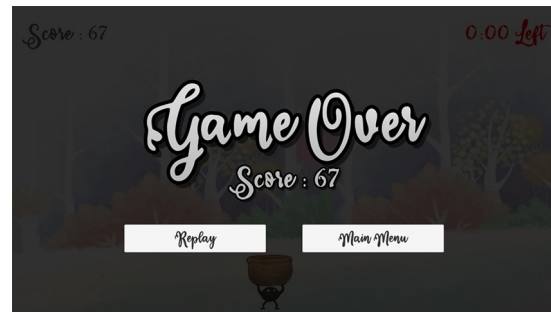
```
Application.Quit();
```


Warning : it doesn't work in the test environment. Only in your final build !

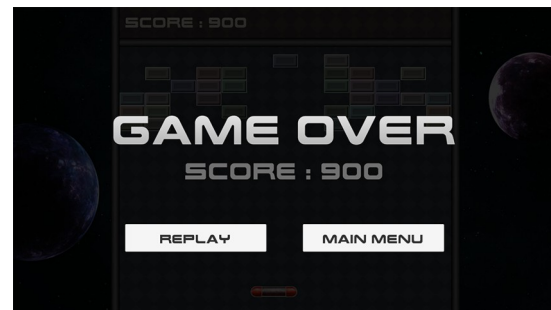


The Game Over Menus

- **The game over screens/menus** are a requirement for all 3 games
In a way they'll act like the « glue » that ties all the part of the app together
- You should have a **Game Over Menu Prefab**. If you want your menus to be slightly different (different fonts and so on), you can create **Prefab variants**
- All say « Game Over », show the score, and offer **two clickable buttons** to either replay the same game, or return to the menu.
This implies that the Game Over Menus are always part of a canvas
- **Code wise, you can use inheritance or interfaces** to manage both the common features and differences of all those menus
- Like everything in this project, **don't hesitate to innovate** and add things to those menus, as long as they improve the experience
Graphical effects, displaying a comment depending on score quality, etc...

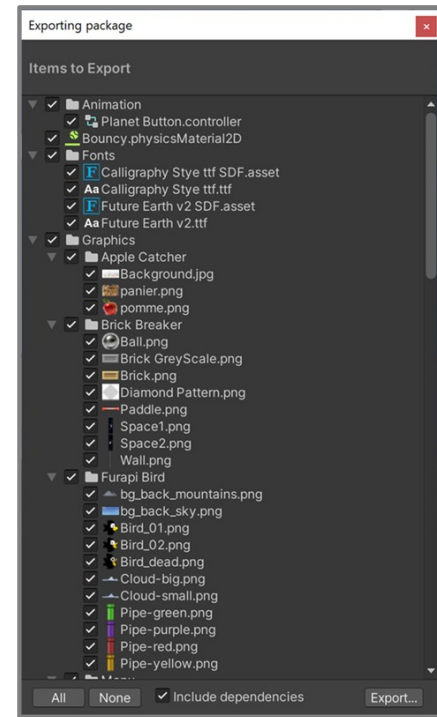


Prefab variants are well suited for prefabs that only have small differences (using them is NOT mandatory, though)



How to Return this project

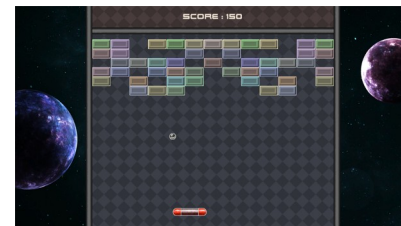
- The projects **should be done in groups of 2 students**. Exceptions will only be allowed in case of a TP group having an odd number of students
- You have to submit a **copy of your Assets folder**. Either as a raw ZIP or as a package. If you go for package, please still place it in a ZIP file anyways
I will use your assets to make my own build of your project and test it
- You have to join a **Readme file within the ZIP**. This Readme file describes the controls of each game, and should mention all the extra features you added
I wouldn't want to miss a special feature or an easter egg in my initial playtest
- This will result in **fairly large files**. Please use moodle to transfer them
They will be rejected as e-mail attachment due to their size, so we have to do this
- Give your ZIP or Asset package a **name that contains both your family names**
Warning : **Asset packages MUST NOT BE RENAMED AFTER CREATION**
So if you messed up the name, make a brand new package instead. Otherwise it won't work.



Try to give (another?) shot at making a package. Test it by importing it in a blank project before sending it to me !

About the Grading

- I like **keeping the assignment « open »**, offering students the ability to innovate and propose interesting features and ideas
In particular with video games. Students often have good ideas for simple but fun additions
- Because of that, doing what this document describes PERFECTLY, with **all the mentioned features and the game over menus**, is only worth about 15 or 16/20
- **4 points are entirely dedicated to your ability to offer extra features**. They can be new game rules, or some neat graphical effect (particles?), etc...
- On the other hand, **do not worry : getting 10/20 is relatively easy** : the menus and the three games just have to be in a playable state with the basic features in
- **Code comments are very important**. Having a description of each method is the bare minimum. And, ideally, I want you to go beyond that requirement by adding some comments inside the methods themselves too !



Godspeed, Unity Creators !



**Good luck for
this final Project !**

Being 3-in-1, it allows me to review everything we did together in one neat project !

Thanks to the power of « recycling », it won't even be that much work to compile everything together.

Since this project has « a little bit of everything », use it as an opportunity to play to your strengths !